



Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness



COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Selecting and Planning the Software Development

Methodology for Your Own Project

Software Development Methodologies

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 7
Output	API flow diagram, request-lifecycle trace, initial API contract, endpoint and validation plan
Maximum Score	80 points

Student / Project Information

Block: A3-IBPH3	Date: September 20, 2026
System Title: RoadWatch: Pothole and Road Damage Detection Using Mobile Dashcam Imagery with a Web-Based Monitoring System	Instructor: Jean Y. Villasin, MIT
Members: RJ Cyrill Jirc L. Monzon, Mae D. Pelipas, Katrice B. Anunciado	

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain the role of the REST API backend and design the request–response contract that links Flutter Web to the PostgreSQL data layer.

Course Outcome Alignment: Supports semester-project server-side planning by connecting the approved Flutter Web client design to an instructor-approved REST API backend and the PostgreSQL data layer developed in IT 108.

Syllabus Assessment: API Flow Diagram; Initial API Contract; online request–response tracing exercise.

Purpose of the Activity

Activity 6 established the Flutter Web UI flow and interface behavior. In Activity 7, your team will define what happens after the Flutter client sends a request: which endpoint receives it, how the server validates and authorizes it, which service/business rule executes, how PostgreSQL data is accessed, and what structured response returns to the client. The API flow diagram and initial API contract will become the technical bridge between your Flutter Web screens and the data layer implemented in IT 108.

Important distinction: This activity designs the server-side flow and API contract. It does not duplicate database modeling, normalization, constraints, indexes, transactions, stored procedures, or other database-design work completed in IT 108.

2. Intended Learning Objectives

- Explain the responsibility of the server-side application and REST API layer in the complete web-system architecture.
- Plan REST endpoints, routes, controllers/handlers, service-layer responsibilities, and conceptual data-access responsibilities for the team's actual system.
- Separate transport concerns, server-side validation, authorization, business logic, and data access into clear responsibilities.
- Define JSON request and response bodies, HTTP methods, status codes, required headers, and structured error responses.
- Trace one important request from the Flutter Web interface through the REST API to PostgreSQL and back to the user interface.
- Produce an initial API contract that is traceable to the approved requirements, use cases, UI flow, and PostgreSQL data domains.

3. Activity Specifications

Mode	Existing semester-project group; all members must participate and understand the API flow.
Suggested Time	One 2-hour laboratory session, with outside time allowed for refinement before submission.
Primary Output	One API Flow Diagram for the team's own system.
Supporting Output	Request-lifecycle trace, endpoint inventory, initial API contract, server-validation/error plan, and conceptual API-to-PostgreSQL mapping.
Recommended Tools	diagrams.net/Draw.io, Figma, Lucidchart, PowerPoint/Canva for diagrams; spreadsheet/Word/Docs for contract tables; Postman/Bruno/Insomnia may be used later for testing.
Scope Rule	Plan only endpoints required by the approved semester-project scope. Do not invent unused features or duplicate the IT 108 database design.

4. Pre-Laboratory Requirements

- Approved system title, problem statement, scope, functional/non-functional requirements, and user roles from Activity 5.
- System-specific Use Case Diagram from Activity 5 and Flutter UI-flow/wireframes from Activity 6.
- Architecture blueprint and prescribed Flutter Web → REST API → PostgreSQL stack from the earlier architecture/technology-stack activities.
- Instructor-approved backend framework/runtime for the REST API.
- The current PostgreSQL table/entity names or data domains from IT 108 for reference only; do not recreate the ERD in this activity.
- At least one critical user workflow that can be traced from the Flutter Web interface to a database operation and back.

5. Part A – Server-Side Responsibility Map

Before drawing the API flow, assign each concern to a clear server-side responsibility. Use the actual technologies and project terms your team plans to implement.

Server-Side Concern	Your Planned Component / Decision	Notes / Evidence
Request entry / routing	GET/POST /detections; POST /sessions; DELETE /sessions/current.	Example: POST /api/reports; route registration
Controller / handler	Handlers parse HTTP input, invoke services and serialize results.	Parses request; calls service; returns HTTP response
Authentication	Planned opaque server session; verify expiry and active account.	Token/session verification; identity extraction
Authorization	Monitor/admin: read. Driver: submit. Server checks role.	Role/permission checks for protected operations
Server-side validation	Pydantic checks type, confidence, GPS, time and record identifier.	Required fields, formats, allowed values, cross-field rules
Service / business logic	Pothole-only acceptance; safe retries; ordered record retrieval.	Approval, recommendation, calculations, workflow rules
Data access	SQLAlchemy repository; bound parameters; transaction for writes.	Repository/query layer; parameterized query or ORM access
PostgreSQL data domain	Existing: public.detections. Proposed: users, auth_sessions.	Reference table/entity names from IT 108 only
External service, if any	None for this trace. On-device ML precedes upload; map tiles are separate.	Maps/GIS, ML service, email/SMS, weather, storage, etc.
Response / error handling	JSON results; safe error envelope; internal server logs only.	Status code, response JSON, safe error message, logging

6. Part B – Request-Lifecycle Tracing Exercise

Select one critical workflow from your own system. Examples: log in, submit a report, create a record, approve a request, retrieve a map/list, update a profile, or request a recommendation.

Selected workflow: UC-06: authenticated review of stored pothole records

Flutter screen / user action: RW-03 Detection Records: open screen or select Refresh

Expected final result shown to the user: Pothole list, empty state, or recoverable error

Step	Actor / Component	Action or Data	Method / Route / Mechanism	Validation / Security	Expected Result
1	Monitoring user	Open Records after login	/#detections	Session required	Loading state
2	Flutter widget	Validate selected controls	Local search/sort choices	Allow known values	Ready to fetch
3	Flutter HTTP client	Send request; no body	GET /detections?limit=50&offset=0	HTTPS; session cookie	API receives GET
4	FastAPI router	Match route; call handler	GET /detections	Only declared method	Handler selected
5	Auth dependency	Verify session and role	users + auth_sessions (planned)	Missing/expired: 401; wrong role: 403	Identity accepted
6	List handler	Parse and validate query	limit 1-100; offset >= 0	Invalid query: 422	Typed parameters
7	Detection service	Apply pothole-only rule	damage_type = Pothole	Read-only authorized view	Repository called
8	SQLAlchemy repo	Read ordered rows	PostgreSQL: detections	Bound query; no client SQL	Rows or empty list
9	Response handler	Serialize permitted fields	200 JSON; safe errors	No credentials or SQL	Structured response
10	Flutter state	Render or recover	List / empty / error / login	401: clear private UI	User sees true state

Tracing Guidance

A strong trace normally includes: (1) user action in Flutter, (2) client-side validation, (3) request creation with method/path/headers/body, (4) route matching, (5) authentication/authorization, (6) server-side validation, (7) service/business-rule execution, (8) PostgreSQL query or transaction, (9) response generation with status + JSON, and (10) Flutter state update and user feedback.

7. Request–Response Trace Check

☐ The trace uses one real use case from Activity 5.

☐ The starting screen exists in the Activity 6 UI flow.

- ☐ The endpoint has a meaningful HTTP method and resource-oriented path.
- ☐ Server-side validation is shown even when Flutter also validates the form.
- ☐ The database step references existing IT 108 data domains rather than recreating the schema.
- ☐ Both success and at least one failure path are identified.

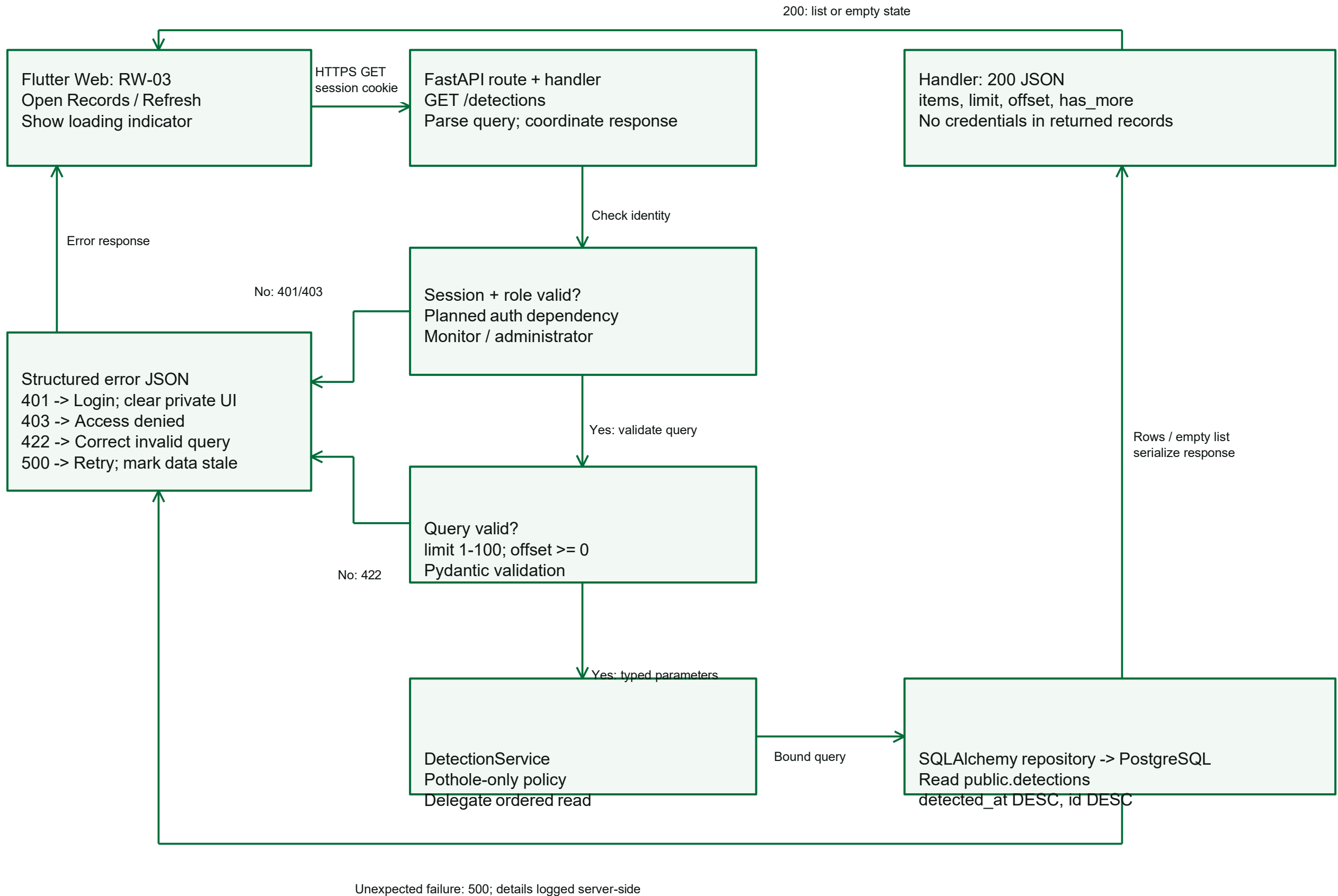
8. Part C – Construct the API Flow Diagram

Create one clean, readable API Flow Diagram showing how a real request travels from Flutter Web through the REST API backend to PostgreSQL, and how the response returns to the interface. The diagram must describe your own project—not a generic sample.

No.	Required Element	What Must Be Shown
1	Flutter Web client	Identify the actual screen/widget or user action that starts the request.
2	Request	Show HTTP method, route/path, HTTPS, headers/token when required, and JSON/body or query parameters.
3	Route / endpoint	Show where the request enters the server-side application.
4	Controller / handler	Show request parsing, response coordination, and delegation to the service layer.
5	Validation & security	Show server-side validation, authentication, authorization, and protected operations where applicable.
6	Service / business logic	Show the rule, calculation, approval, recommendation, or workflow operation performed.
7	Data access	Show repository/query/data-access responsibility without embedding raw database design in the diagram.
8	PostgreSQL	Reference the relevant IT 108 data domain/table/entity names.
9	External service	Include only if the selected workflow actually uses one.
10	Response	Show returned status code, JSON/result, and Flutter state/UI feedback.
11	Error path	Show at least one structured failure path such as invalid input, unauthorized access, not found, conflict, or server error.
12	Legend	Use consistent shapes, arrows, and labels.

Part C - RoadWatch API request-response flow

UC-06 / RW-03 Detection Records | GET /detections | Authentication is a planned extension



Legend: boxes = components/checks; arrows = labeled request, data or response. No external service is needed for this list workflow.

9. Reference Example – Do Not Copy as Your Final Output

The figure below demonstrates the expected level of clarity for a Flutter Web → REST API → PostgreSQL request lifecycle. Replace all labels with the actual endpoint, business logic, security checks, data domains, and response behavior of your own system.

Figure 1: Request-Response Flow in a Flutter Web → REST API → PostgreSQL System

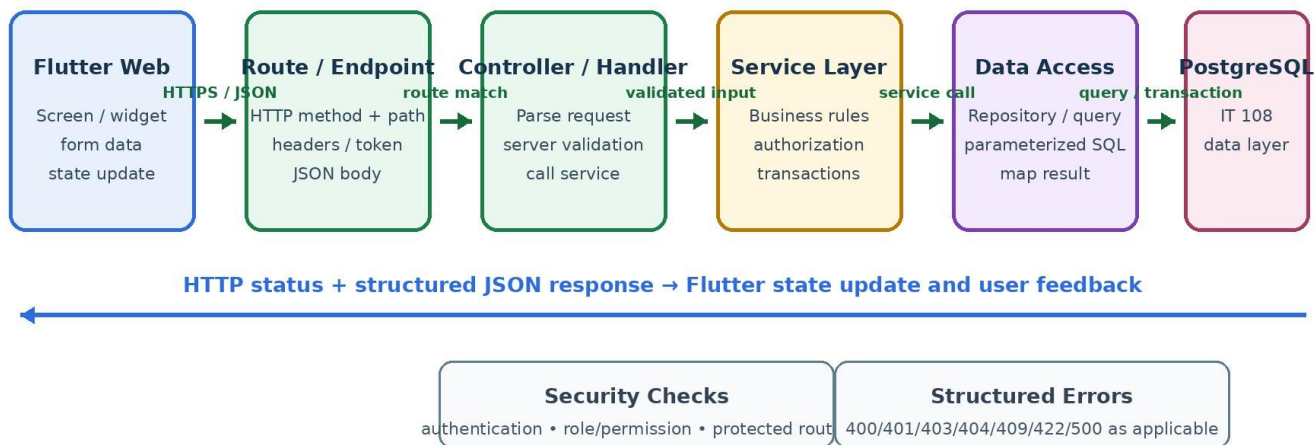


Figure 1. Conceptual request–response path for a Flutter Web client using a REST API and PostgreSQL.

Minimum Flow Rule

Your final diagram must show at least one complete path: Flutter screen/user action → HTTP request → endpoint/route → handler/controller → server validation/security → service/business logic → PostgreSQL or external service → HTTP status + JSON response → Flutter state/UI result. Every arrow on this path must have a meaningful label.

10. Recommended API Flow Labels

Area	Possible Label	What It Represents
Client	Flutter Web screen / widget	User input, event, client-side validation, state update
Transport	HTTPS + REST + JSON	Method, path, headers, body/query parameters
Routing	Endpoint / route	Matches request to server-side handler
Handler	Controller / request handler	Parses request and coordinates response
Service	Service / use-case layer	Business rules, permissions, workflow logic
Data Access	Repository / query layer	Reads/writes PostgreSQL through safe parameterized access or ORM
Security	Auth middleware / guards	Identity, token verification, role/permission checks
Response	HTTP status + JSON	Success result or structured error for Flutter

11. Part D – Semester-Project Endpoint Inventory

List the initial endpoints needed to support the approved core features. Use resource-oriented paths and select methods based on the intended action. Do not create endpoints for features outside the approved scope.

ID	Requirement / Use Case	HTTP Method	Route / Path	Authorized Role(s)	Handler / Service	PostgreSQL Data Domain
EP-01	Login (planned) IFR-15 / UC-08	POST	/sessions	Account holder	create_session / AuthService	users; auth_sessions (planned)
EP-02	FR-11/14; UC-06	GET	/detections	Monitor; admin	list_detections / DetectionService	detections
EP-03	FR-09/10/12/13; UC-05/07	POST	/detections	Driver	create_detection / DetectionService	detections
EP-04	Logout (planned) FR-16 / UC-09	DELETE	/sessions/current	Signed-in user	delete_session / AuthService	auth_sessions (planned)
EP-05	Not required	-	-	-	No extra feature	-
EP-06	Not required	-	-	-	No extra feature	-
EP-07	Not required	-	-	-	No extra feature	-
EP-08	Not required	-	-	-	No extra feature	-

12. REST Method and Route Selection Guide

Method	Typical Purpose	Example	Design Note
GET	Retrieve a resource or collection	GET /api/reports; GET /api/reports/{id}	Should not create side effects.
POST	Create a resource or trigger a non-idempotent operation	POST /api/reports	Usually returns 201 Created when a resource is created.
PUT	Replace a complete resource when the API is designed that way	PUT /api/users/{id}	Use consistently; often idempotent.
PATCH	Partially update a resource	PATCH /api/reports/{id}	Use for partial changes such as status or selected fields.
DELETE	Remove a resource when deletion is allowed	DELETE /api/reports/{id}	Often returns 204 No Content after successful deletion.

Route Naming Reminder

ID	Requirement / Use Case	HTTP Method	Route / Path	Authorized Role(s)	Handler / Service	PostgreSQL Data Domain
Prefer nouns/resources in paths, keep naming consistent, avoid exposing raw database table operations directly, and make endpoint names understandable from the system's domain. Exact conventions may vary with the selected backend framework, but the contract must remain clear and consistent.						

13. Part E – Initial API Contract

Document at least FOUR core endpoints from the inventory. Two of the four must be written in full detail using the contract template below; the other two may be summarized in the endpoint inventory if time is limited during the laboratory period.

Contract ID / Feature	EP-02 / Retrieve pothole records
HTTP Method + Route	GET /detections
Purpose / Related Use Case	FR-11, FR-14; UC-06; RW-03 Detection Records. Also supplies map, overview, analytics and activity.
Authorized Role(s)	monitoring_user or administrator; authorized shared pothole dataset.
Required Headers	Accept: application/json; Cookie: roadwatch_session=<opaque value>. HTTPS. No request Content-Type needed.
Path / Query Parameters	No path parameters. Optional limit: integer 1-100, default 50; offset: integer >=0, default 0.
Request JSON Body	No request body. Example: GET /detections?limit=50&offset=0
Server-Side Validation	Verify session and role; validate query; fixed Pothole filter; order detected_at DESC, id DESC.
Success Response	200: {"items":[],"limit":50,"offset":0,"has_more":false}. Each item has id (integer), damage_type (string), confidence (number), latitude/longitude (number/null), detected_at (ISO-8601 string), client_detection_id (string).
Error Responses	401 AUTH_REQUIRED; 403 FORBIDDEN; 422 VALIDATION_ERROR; 500 INTERNAL_ERROR. All use: {"error":{"code":"AUTH_REQUIRED","message":"Please log in.,"details":{}}} (example 401).
PostgreSQL Data Domain	public.detections (existing); users and auth_sessions (planned access checks).

14. Contract Quality Rules

- The contract must be understandable without looking at the implementation code.
- The route and HTTP method must agree with the feature and request-lifecycle trace.
- Client-side validation from Activity 6 must be repeated or strengthened on the server when the rule protects data integrity or access.
- Do not return passwords, tokens, secrets, internal stack traces, raw SQL, or unnecessary private fields in responses.
- Use consistent field names and JSON types across requests and responses.
- Document expected failure cases instead of treating every failure as a generic 500 error.

13. Part E – Initial API Contract

Document at least FOUR core endpoints from the inventory. Two of the four must be written in full detail using the contract template below; the other two may be summarized in the endpoint inventory if time is limited during the laboratory period.

Contract ID / Feature	EP-03 / Submit or retry one pothole detection
HTTP Method + Route	POST /detections
Purpose / Related Use Case	FR-09/10/12/13; UC-05/07. Mobile synchronization; not a new web entry form.
Authorized Role(s)	mobile_driver with valid session. Re-login is required before retry if session expires.
Required Headers	Content-Type: application/json; Accept: application/json; session cookie. Browser writes also require trusted Origin.
Path / Query Parameters	No path or query parameters. Stable client_detection_id identifies the queued record.
Request JSON Body	{"client_detection_id":"demo_001","damage_type":"Pothole","confidence":0.8,"detected_at":"2026-09-20T02:00:00Z","latitude":null,"longitude":null}. First four required; GPS pair optional. Types shown by JSON values.
Server-Side Validation	Session/driver role; Pothole only; finite confidence 0-1; ID 1-100 nonblank chars; timestamp with zone; GPS both null or lat -90..90/lon -180..180. Reject extra fields. Same ID + same payload: return saved row; changed payload: 409.
Success Response	201 new / 200 exact retry: {"item":<saved record>,"duplicate":false/true}. item returns all request fields plus server id (integer); omitted GPS becomes null. Field types match EP-02. Retry retains the original id.
Error Responses	400 BAD_JSON; 401 AUTH_REQUIRED; 403 FORBIDDEN; 409 ID_CONFLICT; 415 UNSUPPORTED_MEDIA_TYPE; 422 VALIDATION_ERROR; 500 INTERNAL_ERROR. Error JSON uses the EP-02 code/message/details envelope.
PostgreSQL Data Domain	public.detections. Atomic insert/check by client_detection_id. Existing uniqueness constraint must be verified before implementation.

14. Contract Quality Rules

- The contract must be understandable without looking at the implementation code.
- The route and HTTP method must agree with the feature and request-lifecycle trace.
- Client-side validation from Activity 6 must be repeated or strengthened on the server when the rule protects data integrity or access.
- Do not return passwords, tokens, secrets, internal stack traces, raw SQL, or unnecessary private fields in responses.
- Use consistent field names and JSON types across requests and responses.
- Document expected failure cases instead of treating every failure as a generic 500 error.

13. Part E – Initial API Contract

Document at least FOUR core endpoints from the inventory. Two of the four must be written in full detail using the contract template below; the other two may be summarized in the endpoint inventory if time is limited during the laboratory period.

Contract ID / Feature	EP-01 / Login (planned addition)
HTTP Method + Route	POST /sessions
Purpose / Related Use Case	Proposed FR-15 / UC-08; proposed RW-07 Login. Establish identity before viewing the dashboard.
Authorized Role(s)	Unauthenticated account holder. No public self-registration in this scope.
Required Headers	Content-Type: application/json; Accept: application/json. HTTPS; trusted Origin for browser request.
Path / Query Parameters	No path or query parameters.
Request JSON Body	{"username":"demo_monitor","password":"<entered password>"}; both required strings; username 1-100 chars, password 1-256 chars.
Server-Side Validation	Reject malformed/extra fields; verify password hash and active account; rate-limit attempts; role comes from stored account, never the client.
Success Response	200: {"user":{"id":1,"username":"demo_monitor","role":"monitoring_user"}}. Set-Cookie: roadwatch_session=<opaque>; Secure; HttpOnly; SameSite=Strict; Path=/; Max-Age=1800. Store session hash/expiry server-side.
Error Responses	400 BAD_JSON; 401 INVALID_CREDENTIALS; 403 ORIGIN_DENIED; 415; 422; 429 RATE_LIMITED; 500. Use EP-02 error envelope. Generic credential failure; 429 includes Retry-After. No password/session value in JSON.
PostgreSQL Data Domain	Proposed users and auth_sessions. No account/session table was evidenced in the screenshot. IT 108 extension required.

14. Contract Quality Rules

- The contract must be understandable without looking at the implementation code.
- The route and HTTP method must agree with the feature and request-lifecycle trace.
- Client-side validation from Activity 6 must be repeated or strengthened on the server when the rule protects data integrity or access.
- Do not return passwords, tokens, secrets, internal stack traces, raw SQL, or unnecessary private fields in responses.
- Use consistent field names and JSON types across requests and responses.
- Document expected failure cases instead of treating every failure as a generic 500 error.

13. Part E – Initial API Contract

Document at least FOUR core endpoints from the inventory. Two of the four must be written in full detail using the contract template below; the other two may be summarized in the endpoint inventory if time is limited during the laboratory period.

Contract ID / Feature	EP-04 / Logout (planned addition)
HTTP Method + Route	DELETE /sessions/current
Purpose / Related Use Case	Proposed FR-16 / UC-09; logout action in the authenticated dashboard.
Authorized Role(s)	Current session holder. An absent/expired session also receives 204 to permit safe repeated logout.
Required Headers	Cookie: roadwatch_session=<opaque value>, if present. HTTPS; trusted Origin for browser request.
Path / Query Parameters	No path/query parameters.
Request JSON Body	No request body.
Server-Side Validation	Validate browser Origin; revoke matching session if present. Never accept a user ID from the client for logout.
Success Response	204 No Content. Expire the session cookie; Flutter clears private state and returns to Login.
Error Responses	403 ORIGIN_DENIED; 500 INTERNAL_ERROR; EP-02 error envelope. On failure clear local private UI, but do not claim server revocation succeeded.
PostgreSQL Data Domain	Proposed auth_sessions; users association only. Does not delete detections or user accounts.

14. Contract Quality Rules

- The contract must be understandable without looking at the implementation code.
- The route and HTTP method must agree with the feature and request-lifecycle trace.
- Client-side validation from Activity 6 must be repeated or strengthened on the server when the rule protects data integrity or access.
- Do not return passwords, tokens, secrets, internal stack traces, raw SQL, or unnecessary private fields in responses.
- Use consistent field names and JSON types across requests and responses.
- Document expected failure cases instead of treating every failure as a generic 500 error.

15. HTTP Status Codes and Structured Responses

Status	Typical Meaning in the API Contract
200 OK	Successful retrieval/update when a response body is returned.
201 Created	New resource successfully created.
204 No Content	Successful operation with no response body, commonly deletion or update.
400 Bad Request	Malformed request or invalid request syntax/structure.
401 Unauthorized	Authentication is missing or invalid.
403 Forbidden	User is authenticated but lacks permission.
404 Not Found	Requested resource does not exist or is not available to the requester.
409 Conflict	Request conflicts with current resource state, such as duplicate or invalid transition.
422 Unprocessable Content	Request is structurally valid but fails semantic/field validation when this convention is used.
500 Internal Server Error	Unexpected server failure; return a safe message and log details server-side.

16. JSON Request / Response and Error Design

Use valid JSON with consistent field names and types. Your actual contract may use a different structure, but success and error responses must be predictable for the Flutter client.

Example Success	Example Structured Error
<pre>HTTP 201 { "id": 125, "status": "submitted", "message": "Report created" }</pre>	<pre>HTTP 422 { "error": { "code": "VALIDATION_ERROR", "message": "Some fields are invalid", "details": { "location": "Required" } } }</pre>

Status-Code Reminder

The table provides common REST conventions, not a rule that every project must use every status. Select the codes that accurately describe your API behavior, document them in the contract, and apply them consistently.

17. Part F – Server-Side Validation and Error Matrix

For one form or write operation from Activity 6, identify the rules that must be enforced on the server even if Flutter already performs client-side validation.

Field / Rule	Client-Side Check	Server-Side Check	Business / Security Reason	Error Status	Safe Error Message / Code
damage_type	Keep Pothole only	Exact Pothole; reject others	Enforce FR-03/12	422	VALIDATION_ERROR
confidence	Numeric 0-1	Finite number 0-1; no bool	Reject invalid confidence	422	VALIDATION_ERROR
GPS pair	Both present or absent	Both null or valid ranges	No invented location	422	VALIDATION_ERROR
detected_at	Device timestamp	ISO-8601 with timezone	Consistent time order	422	VALIDATION_ERROR
client_detection_id	Keep ID across retries	1-100 chars; exact retry check	One row per queued record	409 / 422	ID_CONFLICT / validation
Session / role	Login before sync	Active session; driver role	Prevent unauthorized writes	401 / 403	AUTH_REQUIRED / FORBIDDEN
Body / fields	JSON; declared keys	Correct media type; reject extras	Protect server-owned id	400 / 415 / 422	BAD_JSON / media / validation

18. Part G – Conceptual API-to-PostgreSQL Mapping

Connect API operations to the existing PostgreSQL data layer from IT 108 at a conceptual level. Do not recreate the ERD, normalization, keys, constraints, or physical schema here.

Endpoint	Service / Business Operation	Data Action	IT 108 Data Domain / Entity	Returned Result
POST /sessions	Verify credentials; create session	Read / create	users; auth_sessions (planned)	User + cookie
GET /detections	Authorize; list potholes	Read	detections	200 items + paging
POST /detections	Validate; save or return retry	Read / insert	detections	201 new / 200 retry
DELETE /sessions/current	Revoke current session	Update/delete session	auth_sessions (planned)	204; clear cookie

Endpoint	Service / Business Operation	Data Action	IT 108 Data Domain / Entity	Returned Result
IT 108 Boundary Reference the approved IT 108 table/entity or data-domain names only to show where the API reads or writes. Do not recreate or redesign the database schema in this Activity 7 manual.				

19. Laboratory Procedure

Phase 1 – Select a critical workflow (10 minutes)

Choose one high-value use case that begins in an Activity 6 Flutter screen and requires the REST API and PostgreSQL.

Phase 2 – Trace the request lifecycle (20 minutes)

Complete Part B from user action to Flutter request, route, validation/security, service logic, data access, response, and UI feedback.

Phase 3 – Draft the API Flow Diagram (25 minutes)

Draw the complete system-specific request/response path. Label methods, paths, data, validation, security checks, results, and errors.

Phase 4 – Plan endpoints (20 minutes)

Build the endpoint inventory for the approved core features. Review method selection, path naming, roles, services, and data domains.

Phase 5 – Write the initial API contract (30 minutes)

Document at least four core endpoints; fully detail two using the contract template.

Phase 6 – Validate responses and errors (10 minutes)

Check JSON bodies, HTTP status codes, headers, field validation, authorization, and structured errors.

Phase 7 – Review API-to-PostgreSQL mapping (10 minutes)

Confirm that the API references the current IT 108 data domains without duplicating database design.

Phase 8 – Final review and submission preparation (15 minutes)

Use the checklist, update the diagram and contract, export required files, and commit documentation to GitHub.

20. Final Quality Checklist

☐ The API Flow Diagram uses the official system title and a real critical workflow.	☐ JSON request/response fields and data types are consistent.
☐ The starting Flutter screen/user action exists in Activity 6.	☐ Success and failure status codes are documented.
☐ At least one complete request-and-response path is visible and labeled.	☐ Errors are structured and do not expose internal or sensitive details.
☐ HTTP methods and route paths are meaningful and consistent.	☐ PostgreSQL references agree with the IT 108 data layer and do not duplicate schema design.
☐ Controllers/handlers and service/business logic are not treated as the same responsibility without explanation.	☐ Endpoint inventory is traceable to approved requirements/use cases.
☐ Server-side validation is included even when Flutter validates the same input.	☐ At least four core endpoints are documented and two are fully detailed.
☐ Authentication and authorization are shown for protected endpoints.	☐ Every group member can explain the selected request lifecycle and API contract.

21. Submission Requirements

- PDF or PNG of the final API Flow Diagram in readable landscape orientation.
- Editable diagram source file (.drawio, .pptx, .fig, or native file of the chosen tool).
- Completed request-lifecycle tracing worksheet.
- Endpoint inventory and Initial API Contract with at least four core endpoints; at least two endpoints must contain full request/response, validation, status-code, and error details.
- Completed server-side validation/error matrix and conceptual API-to-PostgreSQL mapping.
- Commit the files to the project GitHub repository under a folder such as docs/api/ or docs/backend/.
- Submit the repository link and required files through the class submission platform using the naming convention: IT110_LA7_Group##_SystemName.

Academic Integrity and Competency Validation

The API design must represent the team's own approved system and must be consistent with its requirements, use case diagram, Flutter UI flow, and IT 108 data layer. AI tools may be used to clarify concepts or improve documentation, but every endpoint, validation rule, role restriction, request/response field, and flow decision must be reviewed and understood by the team. The instructor may ask any member to trace a request or explain an endpoint without assistance.

22. Scoring Rubric – 80 Points

Criterion	Evidence for Full Credit	Points
API flow correctness and completeness	The diagram accurately traces Flutter → REST API → PostgreSQL → response, including routes, handlers, validation/security, service logic, data access, and error/success paths.	15
Endpoint planning and REST design	Core endpoints are traceable to approved features; methods and resource-oriented paths are appropriate, consistent, and limited to the project scope.	12
Initial API contract completeness	At least four core endpoints are documented and the required detailed contracts clearly define purpose, roles, parameters, headers, request/response bodies, validation, status codes, and errors.	15
HTTP semantics and JSON design	Methods, status codes, request/response fields, JSON types, headers, and result structures are technically coherent and consistently documented.	10
Server-side validation, security, and errors	Validation, authentication, authorization, protected operations, safe error messages, and structured failure responses are clearly planned.	10
Conceptual API-to-PostgreSQL mapping	Endpoints and service operations map correctly to existing IT 108 data domains without duplicating database design.	8
Traceability and implementation readiness	The API plan aligns with Activity 5 requirements/use cases and Activity 6 Flutter screens and can guide subsequent backend implementation.	5
Documentation quality and technical clarity	Tables, labels, terminology, diagrams, and file organization are readable, consistent, and professionally prepared.	5

23. Post-Laboratory Reflection

Which endpoint in your system is the most security-sensitive, and what authentication/authorization checks must occur before its business logic runs?

__POST /sessions is the most sensitive: validate input, rate-limit attempts and verify the stored password hash and active account.
__Protected endpoints then verify the session and permitted role before business logic.

Which validation rule must be enforced on the server even if Flutter already checks it, and why?

__The API must require damage_type = Pothole and confidence within 0-1. A modified client can bypass Flutter checks, so the server
__must protect stored data independently.

Which API response will be the most difficult for the Flutter client to handle correctly, and what UI state should be shown?

__A lost response after a successful upload is difficult: keep the record pending and retry with the same client_detection_id. A
__matching retry returns the existing record; mark synced only after acknowledgment.

Where is the clearest boundary between the controller/handler and service/business-logic responsibilities in your selected workflow?

__The handler owns HTTP parsing, status codes and JSON. The service owns the pothole-only policy, authorized retrieval and retry
__rules; the repository owns database access.

What API contract detail must your team finalize before beginning the next server-side implementation activity?

__Finalize the account/session extension, role provisioning, GPS-null convention and response shapes, then verify the
__client_detection_id uniqueness constraint and align Activities 5 and 6 before coding.